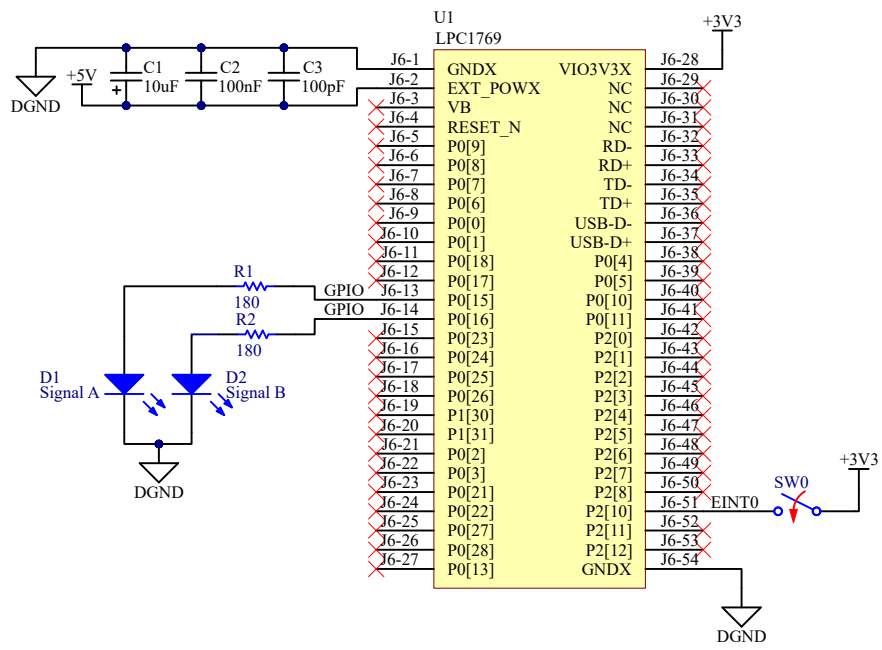


```

1  /*=====
2  * @file    Ejercicio_4_3.c
3  *
4  * @brief    Diseñar e implementar en C un generador de dos señales cuadradas de
5  *           50 Hz por software utilizando el temporizador SysTick, con control de
6  *           desfase (phase shift) dinámico entre ambas señales mediante la
7  *           interrupción externa EINT0, por flanco ascendente.
8  *
9  * @details  - 1. Generar por el pin P0.15 la Señal A (referencia) como una onda
10 *             cuadrada con un período de 20 ms (50 Hz) y un ciclo de trabajo
11 *             (duty cycle) del 50% (10 ms en ALTO, 10 ms en BAJO).
12 *             - 2. Generar por el pin P0.16 la Señal B, idéntica en frecuencia y
13 *             duty cycle, pero con un desfase temporal respecto a la Señal A.
14 *             - 3. Mediante la pulsación de un switch conectado a EINT0,
15 *             rotar cíclicamente el desfase de la Señal B en 4 estados:
16 *             0° (0 ms), 90° (5 ms), 180° (10 ms) y 270° (15 ms).
17 *             - 4. Configurar el SysTick con una base de tiempo fija de 1 ms,
18 *             considerando CCLK = 100 MHz.
19 *
20 * @note     Estado 0° (Desfase: 0 ms)
21 *           Ambas señales coinciden perfectamente en fase.
22 *           Señal A (P0.15) ...
23 *           Señal B (P0.16) ...
24 *
25 *           Estado 90° (Desfase: 5 ms)
26 *           La Señal B conmuta a nivel ALTO 5 ms después que la Señal A.
27 *           Señal A (P0.15) ...
28 *           Señal B (P0.16) ...
29 *
30 *           Estado 180° (Desfase: 10 ms)
31 *           Las señales están en oposición de fase (cuando A está en ALTO,
32 *           B está en BAJO).
33 *           Señal A (P0.15) ...
34 *           Señal B (P0.16) ...
35 *
36 *           Estado 270° (Desfase: 15 ms)
37 *           La Señal B conmuta a nivel ALTO 15 ms después que la Señal A
38 *           (o 5 ms antes del siguiente ciclo).
39 *           Señal A (P0.15) ...
40 *           Señal B (P0.16) ...
41 *
42 * @author    Ing. E. Migliore
43 *
44 * @version 1.0
45 *
46 * =====*/

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9  #define NVIC_PRIO_BITS      5U          /* Bits físicos de prioridad (LPC1769) */
10
11  /* Identificadores de Prioridad de IRQ */
12  #define SYSTICK_PRIORITY    //COMPLETAR
13  #define EINT0_PRIORITY      //COMPLETAR
14
15  /* Identificadores de IRQ */
16  #define SYSTICK_IRQ         SysTick_IRQn
17  #define EINT0_IRQ           EINT0_IRQn
18
19  /* Valores de recarga para SysTick */
20  #define N_TICKS_1MS        //COMPLETAR
21
22  #define SYSTICK_CTRL_ON     //COMPLETAR
23  #define SYSTICK_CTRL_OFF   //COMPLETAR
24
25  #define PERIOD_STEPS        20U          /* 20 pasos de 1 ms = Período de 20 ms
26  #define HALF_PERIOD_STEPS   10U          /* 10 ms Alto / 10 ms Bajo (50% Duty Cycle)
27
28  /* Opciones de desfasaje */
29  #define SHIFT_0_DEG         0U           /* 0 ms de retraso (0°)
30  #define SHIFT_90_DEG        5U           /* 5 ms de retraso (90°)
31  #define SHIFT_180_DEG       //COMPLETAR /* 10 ms de retraso (180°)
32  #define SHIFT_270_DEG       //COMPLETAR /* 15 ms de retraso (270°)
33
34  /*=====
35  * DEFINICIÓN DE VARIABLES GLOBALES
36  *=====*/
37  volatile uint8_t shiftSteps = SHIFT_0_DEG;
38
39  /*=====
40  * PROTOTIPOS DE FUNCIONES
41  *=====*/
42  static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number);
43
44  void cfgGPIO(void);
45  void cfgIntExt(void);
46  void cfgSysTick(uint32_t nTicks, uint32_t cTicks);
47
48  /*=====
49  * FUNCIÓN PRINCIPAL
50  *=====*/
51  int main(void)
52  {
53      /*-----
54      * INICIALIZACIÓN DEL MCU
55      *-----*/
56      cfgGPIO();
57      cfgIntExt();
58      cfgSysTick(N_TICKS_1MS, SYSTICK_CTRL_ON);
59
60      /*-----
61      * BUCLE PRINCIPAL
62      *-----*/
63      while(1){}
64
65      return 0;
66  }
67
68  /*=====
69  * DEFINICIONES DE FUNCIONES

```

```

70  *=====*/
71  /*****
72  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
73  *
74  * @param   <irq_number>      Número de IRQ en la tabla del vector.
75  * @param   <priority_number> Nivel de prioridad del vector.
76  *
77  * @return  None
78  *****/
79  static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
80  {
81      //COMPLETAR
82  }
83
84  /*****
85  * @brief   Configura las direcciones GPIO, modos de función y resistencias.
86  *
87  * @details - P0.15 - P0.16: Salidas digitales.
88  *          - P2.10: Función EINT0 con Pull-Down interno.
89  *
90  * @param   None
91  *
92  * @return  None
93  *****/
94  void cfgGPIO(void)
95  {
96      //COMPLETAR
97
98      return;
99  }
100
101  /*****
102  * @brief   Configura las interrupciones externas EINT0.
103  *
104  * @details - EINT0 (P2.10): Detección por Flanco Ascendente.
105  *          - Limpia banderas pendientes y asigna prioridades en el NVIC.
106  *
107  * @param   None
108  *
109  * @return  None
110  *****/
111  void cfgIntExt(void)
112  {
113      //COMPLETAR
114
115      return;
116  }
117
118  /*****
119  * @brief   Configura el módulo SysTick Timer
120  *
121  * @param   <nTicks> Valor de recarga del registro LOAD.
122  * @param   <cTicks> Registro de control CTRL (Habilitación/Reloj/IRQ).
123  *
124  * @return  None
125  *****/
126  void cfgSysTick(uint32_t nTicks, uint32_t cTicks)
127  {
128      //COMPLETAR
129
130      return;
131  }
132
133  /*****
134  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector EINT0.
135  *
136  * @details Conmuta secuencialmente el desfase de la Señal B entre 0°, 90°,
137  *          180° y 270°.
138  *

```

```

139 * @param    None
140 *
141 * @return    None
142 *****/
143 void EINT0_IRQHandler(void)
144 {
145     switch(shiftSteps)
146     {
147         case SHIFT_0_DEG:
148             shiftSteps = SHIFT_90_DEG;
149             break;
150         case SHIFT_90_DEG:
151             shiftSteps = SHIFT_180_DEG;
152             break;
153         case SHIFT_180_DEG:
154             shiftSteps = SHIFT_270_DEG;
155             break;
156         case SHIFT_270_DEG:
157             shiftSteps = SHIFT_0_DEG;
158             break;
159     }
160
161     //COMPLETAR
162
163     return;
164 }
165
166
167 /*****
168 * @brief      Rutina de Servicio de Interrupción (ISR) para el vector SysTick.
169 *
170 * @details    Genera ambas señales cuadradas actualizando los pines P0.15 y P0.16
171 *             aplicando el retraso temporal configurado.
172 *
173 * @param      None
174 *
175 * @return     None
176 *****/
177 void SysTick_Handler(void)
178 {
179     static uint8_t counterA = 0;
180     uint8_t counterB;
181
182     // Generación de la Señal A (Referencia)
183     if (counterA < HALF_PERIOD_STEPS)
184     {
185         LPC_GPIO0->FIOSET = (1 << 15);
186     }
187     else
188     {
189         LPC_GPIO0->FIOCLR = (1 << 15);
190     }
191
192     // Cálculo del índice temporal con retardo para la Señal B (Desfasada)
193     counterB = (counterA + PERIOD_STEPS - shiftSteps) % PERIOD_STEPS;
194
195     if (counterB < HALF_PERIOD_STEPS)
196     {
197         LPC_GPIO0->FIOSET = (1 << 16);
198     }
199     else
200     {
201         LPC_GPIO0->FIOCLR = (1 << 16);
202     }
203
204     // Incrementa la base de tiempo
205     counterA++;
206     if (counterA >= PERIOD_STEPS)
207     {

```

```
208         counterA = 0;
209     }
210
211     return;
212 }
```